

Su RAG puede buscar mejor.

La semana pasada montaron el RAG ingenuo y anotaron dónde falla su búsqueda. Hoy: tres familias de técnicas para arreglar exactamente esos fallos — hybrid search, reranking y query transformation — y la regla de siempre: cada una se gana su lugar con el delta en SU harness.

HYBRID SEARCH

RERANKING

QUERY TRANSFORMATION

La sesión de hoy.

 Recap: resolvemos las dudas del formulario de salida

 Los problemas del RAG ingenuo (sus consultas fallidas)

 Hybrid search: BM25 + denso, fusión con RRF

 Reranking con cross-encoders

 Query transformation: multi-query, HyDE, step-back

 Medir el delta con su harness • hacia S09

 Lab

Primero,
sus dudas.

Las métricas, en palabras sencillas.

La duda más repetida del formulario: ¿BLEU vs ROUGE?, ¿y METEOR?, ¿y BERTScore? La chuleta definitiva:

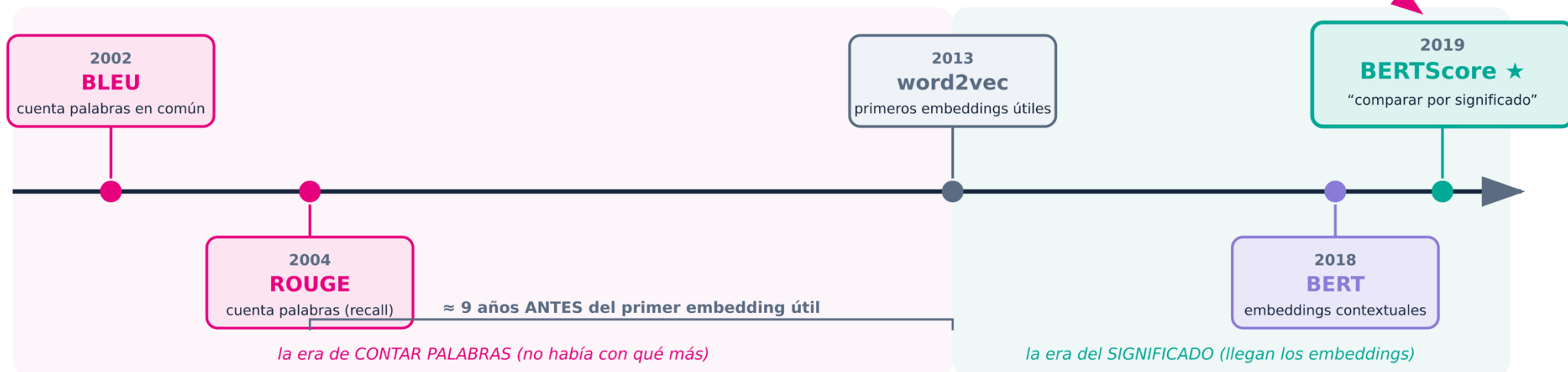
	BLEU	de lo que DIJO el modelo, ¿cuánto está en la referencia? (precisión)	traducción · castiga inventar de más
	ROUGE	de la REFERENCIA, ¿cuánto recuperó el modelo? (recall)	resumen · castiga omitir
	METEOR	como BLEU, pero acepta sinónimos y raíces ("vacunar"≈"vacunación")	un poco más justa, misma familia
	BERTScore	compara SIGNIFICADO con embeddings, no palabras exactas	rescata paráfrasis · no ve la verdad
	Perplexity	¿qué tan poco se sorprende el modelo con el texto? = $\exp(\text{cross-entropy})$, el training loss de M1	fluidez, sin referencia · no mide verdad

¿Cuál es "buena para mi dominio"? Ninguna sola: por eso su harness combina una automática + el juez + sus gold. La métrica no se elige por moda sino por el modo de fallo que les duele.

"¿Por qué BLEU/ROUGE no usan embeddings, si ya existían?"

Respuesta corta: no existían todavía. Mira las fechas.

tu idea YA la hicieron:
se llama BERTScore



¿Y por qué BLEU/ROUGE siguen vivas en 2026?



BARATAS

sin GPU, sin modelo:
corren en milisegundos



REPRODUCIBLES

mismo texto → mismo número,
hoy y en 10 años



COMPARABLES

el estándar de 20 años
de papers y experimentos

Moraleja: "vieja" no significa inútil — significa saber qué NO ve (el significado) y complementarla.

"¿Por qué BLEU/ROUGE no usan embeddings, si ya existían?"

Excelente pregunta — con respuesta en tres partes:

- **Historia: los embeddings ÚTILES aún no existían.** BLEU es de 2002 y ROUGE de 2004; word2vec llega en 2013 y los embeddings contextuales (BERT) en 2018. Cuando se diseñaron, comparar palabras exactas era lo computable.
- **Y cuando llegaron... alguien hizo exactamente lo que propones:** eso ES BERTScore (2019): la misma idea de comparar candidato vs referencia, pero en el espacio de embeddings. Tu pregunta reinventa su motivación — señal de que entendiste el problema.
- **¿Y por qué BLEU/ROUGE siguen vivas?** Son baratas, rapidísimas, deterministas y 100% reproducibles (mismo texto → mismo número, sin GPU ni modelo de por medio). Para comparar contra 20 años de literatura, siguen siendo el estándar.

Moraleja: las métricas también tienen historia y trade-offs. "Vieja" no significa inútil — significa que hay que saber qué NO ve (significado) y complementarla.

Detrás de todo número de benchmark hay 3 preguntas



El número no miente — pero solo responde lo que su regla midió, sobre preguntas que quizá ya vio,
de un examen que no es el suyo. Por eso el harness de M2 existe.

Benchmarks: ¿memoriza o generaliza?

“¿Cómo reconozco si un benchmark refleja generalización y no memorización?”

No se puede saber solo con el número. Tres pruebas: (1) reformular las preguntas — si el puntaje se desploma, era memoria; (2) buscar solapamiento de n-gramas entre test y datos de entrenamiento; (3) canary strings. Y la defensa definitiva: un eval set propio que nunca estuvo en internet.

“¿Qué es MMLU?”

Un examen público de opción múltiple con ~57 materias académicas. Mide conocimiento general con accuracy — no mide su dominio, su idioma ni tareas generativas. Útil para preseleccionar modelos, nunca para decidir.

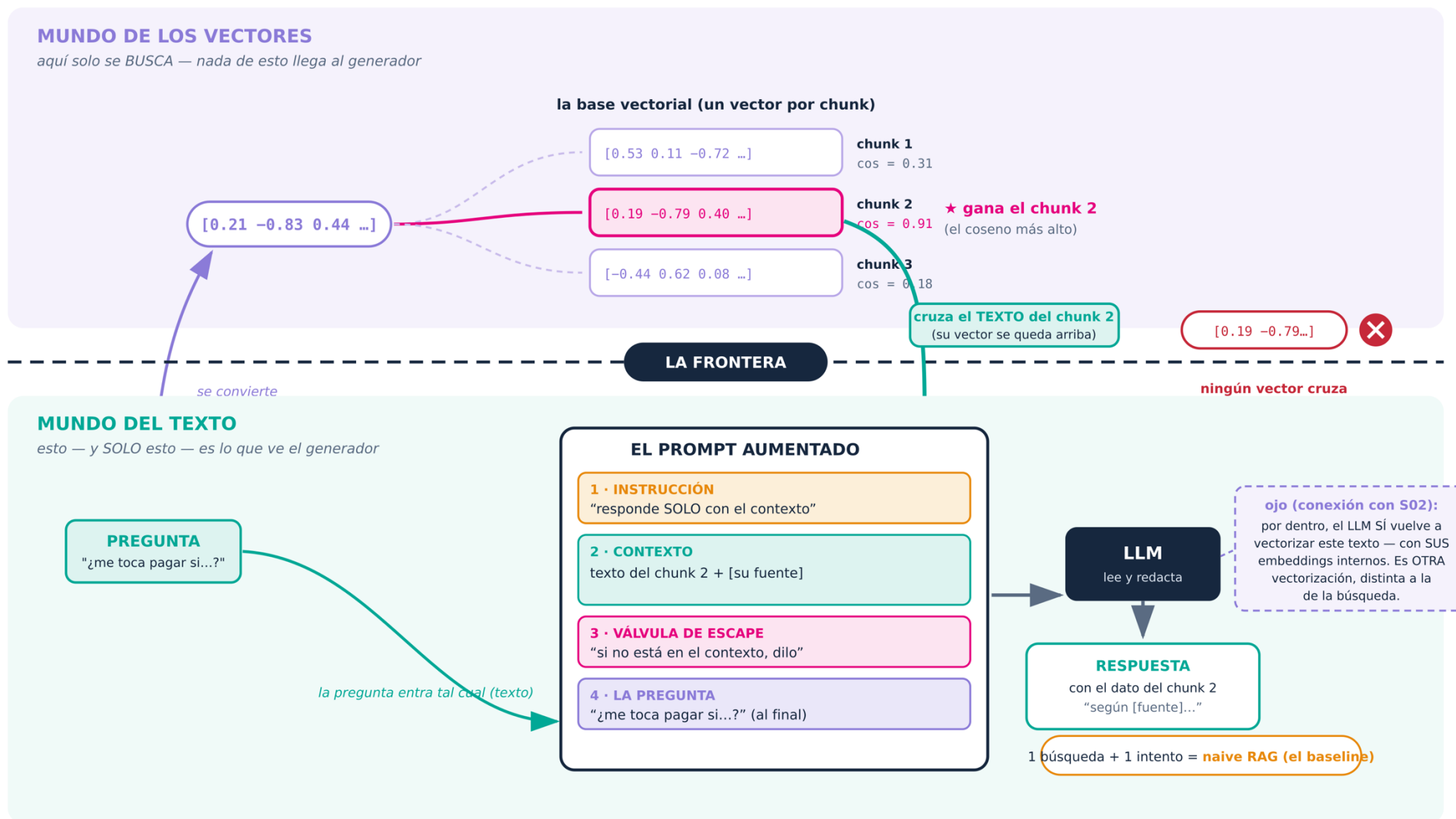
“No me quedaron claras las reglas de puntuación.”

Todo benchmark se reduce a UNA regla mecánica: exact-match (¿coincide exacto?), opción múltiple (¿acertó la letra?) o pass@k (de k intentos de código, ¿alguno pasa los tests?). "85%" siempre es 85% según ALGUNA regla

“¿Cómo decido qué es bueno para MI dominio? ¿Qué era el harness?”

Exactamente para eso construyeron el harness en M2: SU eval set (gold + adversariales) + 3 dimensiones (embeddings, juez, aciertos). Esa es la única medida hecha para su caso — los benchmarks públicos no compiten con eso.

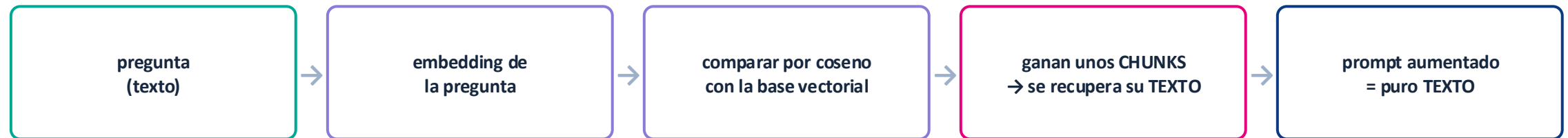
El RAG tiene DOS mundos — y los vectores no cruzan la frontera



Los vectores BUSCAN arriba • el TEXTO viaja abajo • el generador nunca RECIBE un embedding — los suyos los fabrica por dentro

El malentendido clave: los embeddings NO viajan al prompt.

Alguien preguntó: "¿cómo se UNEN los embeddings del prompt con los de mi base para crear el prompt aumentado?" — y la respuesta corrige un malentendido común:



Los embeddings SOLO se usan para BUSCAR (comparar pregunta vs chunks). Nada de vectores se "une" ni entra al prompt: lo que viaja al prompt es el TEXTO de los chunks ganadores. El modelo generador nunca ve un embedding.

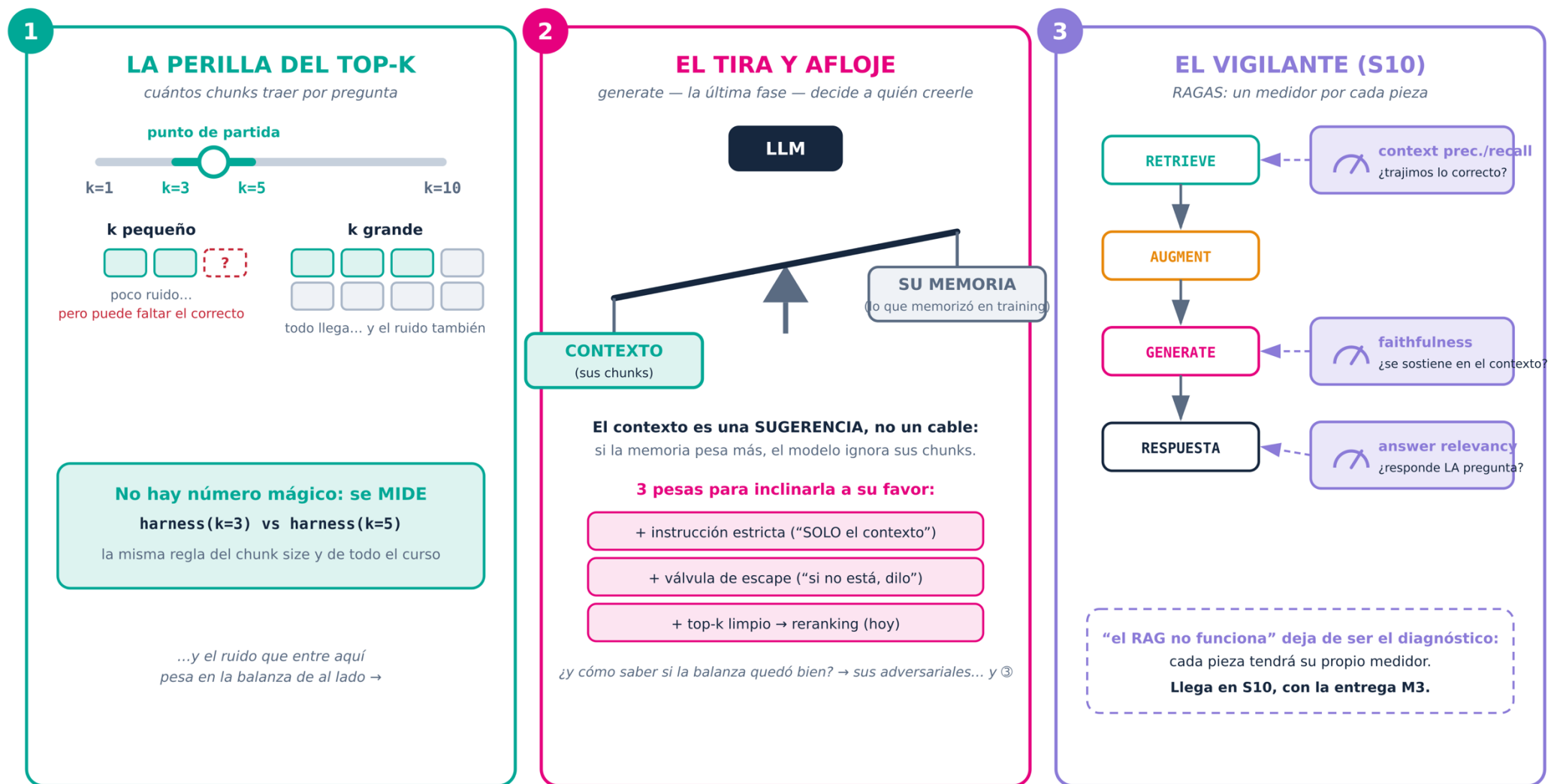
“¿La estructura del prompt aumentado?”

Cuatro partes, en orden: instrucción ("responde SOLO con el contexto") + contexto (el texto de los k chunks, cada uno con su fuente) + válvula de escape ("si no está, dilo") + la pregunta del usuario al final.

“¿La evaluación de los documentos indexados? ¿Qué era "naive RAG"?”

"Naive" = ingenuo: una búsqueda, un intento, sin trucos — el baseline de hoy. Y la indexación se evalúa indirectamente: si el harness baja o el retrieval trae ruido, se revisa chunking y embeddings (hoy) y con RAGAS por componente (S10).

Las perillas del RAG: cuánto traer, a quién creerle, quién lo vigila



Top-k, el modelo rebelde, y lo que viene.

“¿Cómo sé qué top-k manejar?”

Como el chunk size: no hay número mágico — se MIDE. k pequeño (2-3): poco ruido pero puede faltar el chunk correcto; k grande (8-10): más cobertura pero más ruido y más costo. Prueben k=3 y k=5 con su harness y comparen. Punto de partida: 3-5.

“¿Por qué el modelo podría ignorar el RAG y saltárselo?”

Porque el contexto es una SUGERENCIA, no un cable: el modelo pondera contexto vs su memoria, y si memorizó fuerte otra cosa (o el chunk llega débil/ruidoso), gana la memoria. Mitigaciones: instrucción estricta + válvula de escape, menos ruido en el top -k (reranking, hoy), y medirlo con los adversariales.

“¿La última fase del RAG?”

Generate: el prompt aumentado (instrucción + chunks + pregunta) entra al LLM y este redacta la respuesta leyendo el contexto — y citando la fuente si se lo pedimos.

“¿Qué es RAGAS?”

La metodología de evaluación específica de RAG: mide por componente — faithfulness (¿la respuesta se sostiene en el contexto?), context precision/recall (¿recuperamos lo correcto?) y answer relevancy. La vemos a fondo en S10 con la entrega M3.

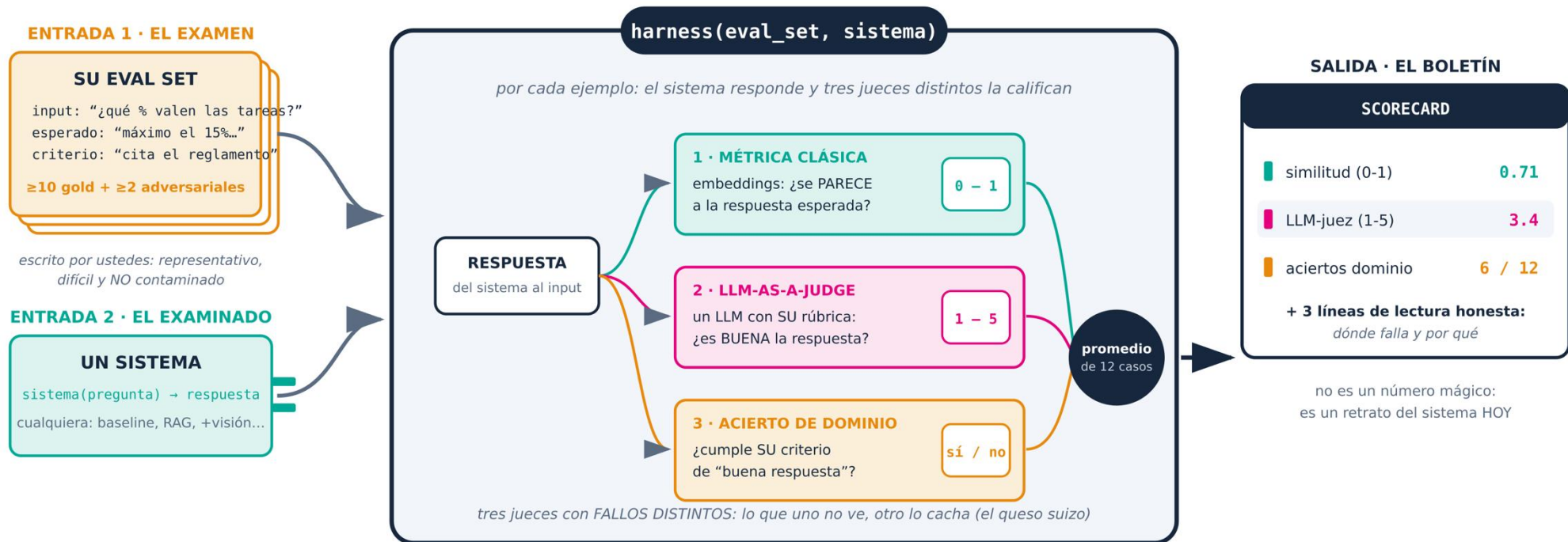
“¿Qué función cumple METEOR?”

Es BLEU con flexibilidad: acepta sinónimos y palabras de la misma raíz antes de contar coincidencias (slide 4). Sigue midiendo o forma, no verdad.

Slide anti dissociation

¿Qué es el harness? La máquina de medir que ustedes construyeron

entra un examen y un sistema · sale un boletín — y la máquina no cambia nunca

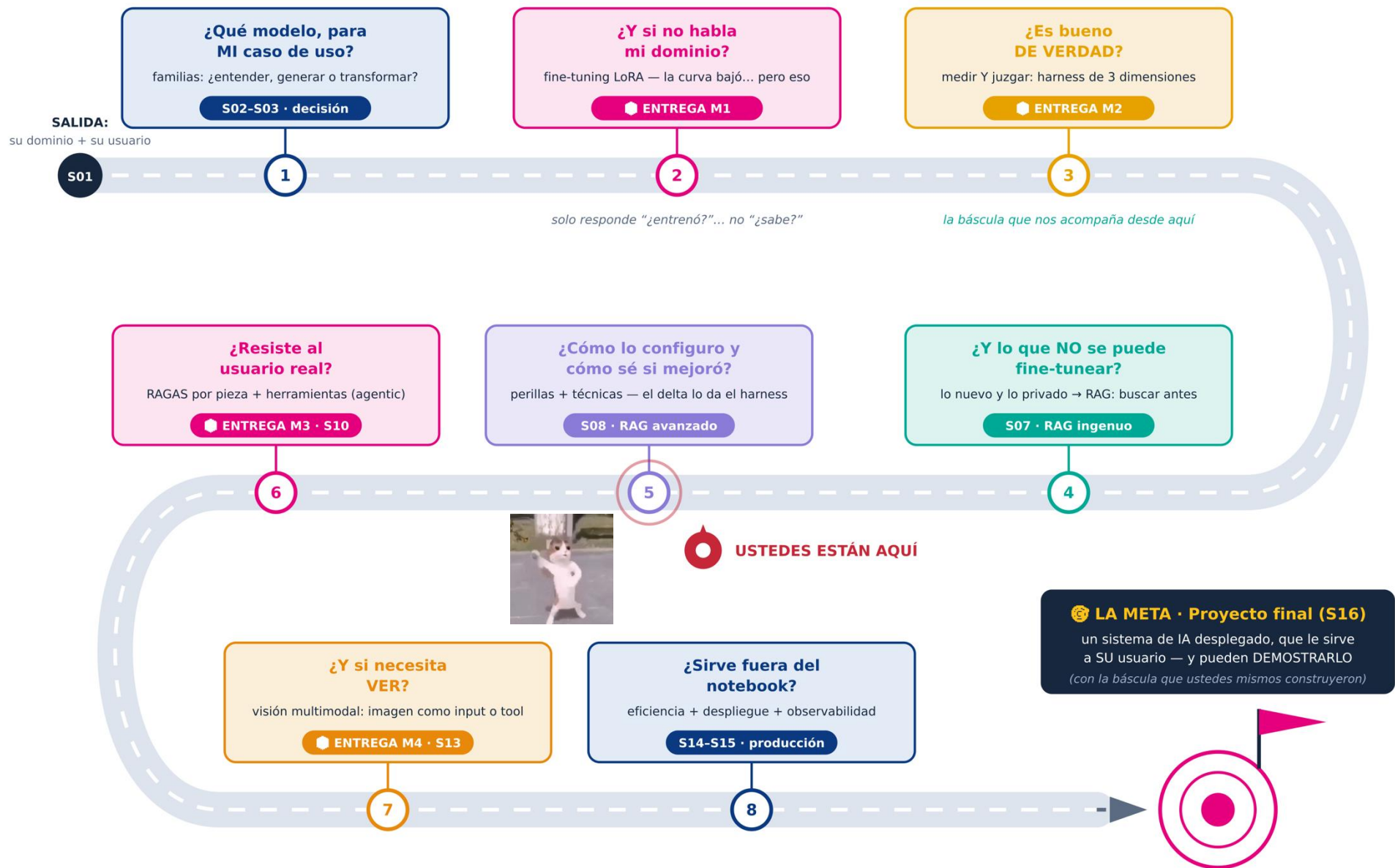


eval set = el examen · harness = el profesor que lo aplica y califica, siempre igual · scorecard = el boletín

Dónde estamos?

El mapa del semestre: cada pregunta nos empujó a la siguiente

no estamos coleccionando temas — estamos construyendo UNA aplicación, respondiendo sus preguntas en orden



Ninguna entrega se bota: M1 + M2 + M3 + M4 + producción = el proyecto final. Cada parada viaja con ustedes hasta la meta.

Los problemas del ingenuo.

Las consultas fallidas que trajeron son el mapa de hoy: cada fallo tiene su técnica.

Cada fallo de retrieval tiene su remedio.



El corpus usa términos EXACTOS

buscó "Ley 1801" y la semántica trajo "normas de convivencia" en vez del artículo literal

HYBRID SEARCH



El top-k llega con RUIDO

los chunks correctos existen pero llegan revueltos con vecinos temáticos que no responden

RERANKING



La PREGUNTA es el problema

consulta coloquial, ambigua o doble ("¿vacuna y desparasitación para viajar?") que no matchea bien

QUERY TRANSFORMATION

Antes de aplicar una técnica, diagnostiquen CUÁL fallo tienen: la técnica equivocada agrega costo y latencia sin mover el harness.

Dos buscadores mejor que uno.

BM25 (palabras exactas) + búsqueda densa (significado), fusionados con Reciprocal Rank Fusion.

BM25: el buscador de palabras exactas.

BM25 es el algoritmo clásico de los buscadores (pre-embeddings): puntúa un chunk según cuántas veces contiene las palabras de la consulta, premiando las palabras **raras** (si "braquicefálico" aparece, pesa mucho más que "el") y normalizando por longitud del documento.

¿Por qué sigue vivo en 2026? Porque hay consultas donde lo EXACTO es lo que importa:

- **Códigos y siglas:** "Ley 1801", "SKU-4432", "E. coli O157" — el embedding los difumina; BM25 los clava.
- **Nombres propios y términos técnicos:** la palabra exacta del dominio a veces ES la consulta ("ivermectina", "roya del café").
- **Vocabulario que el modelo de embeddings no conoce:** jerga local o términos nuevos quedan mal vectorizados — pero se matchean literal.

La lección de ingeniería: lo nuevo no siempre reemplaza a lo viejo — a veces lo complementa. Denso ve significado; BM25 ve superficie. Sus fallos NO están correlacionados: por eso la mezcla paga.

Dos buscadores, fortalezas opuestas

	 DENSO busca por SIGNIFICADO	 BM25 busca por PALABRAS EXACTAS
CONSULTA 1 · coloquial (paráfrasis) consulta: <i>"¿puedo llevar mi perro chato en bodega?"</i> el chunk correcto dice: ...las razas braquicefálicas no viajan en bodega...	 "chato" ≈ "braquicefálico" lo entiende sin compartir ni una palabra	 0 palabras en común para BM25 esta consulta es invisible
CONSULTA 2 · exacta (código, artículo, sigla) consulta: <i>"multa del artículo 140 de la Ley 1801"</i> el chunk correcto dice: Ley 1801, artículo 140: multa tipo 4 por...	 trae los artículos VECINOS (139, 141... "temáticamente cercaños", pero no ESE)	 "Ley 1801" = "Ley 1801" "artículo 140" = "artículo 140" lo clava literal

Cada uno falla donde el otro acierta → no se elige uno: se COMBINAN (hybrid search)

sus fallos no están correlacionados — por eso la mezcla paga

Pero ojo, el problema técnico:

BM25 = 12.7

VS

coseno = 0.83

→ escalas incomparables: **NO se promedian puntajes**

la salida (siguiente slide): fusionar por PUESTOS — 1º es 1º en cualquier escala. Eso es RRF.

Denso vs. BM25: fortalezas opuestas.

DENSO (EMBEDDINGS)

Gana con paráfrasis

Consulta: "¿puedo llevar mi perro chato en bodega?"

Encuentra el chunk de "razas braquicefálicas" sin compartir palabras.
BM25 aquí no ve NADA.

BM25 (LÉXICO)

Gana con lo exacto

Consulta: "multa del artículo 140 de la Ley 1801"

Matchea el artículo literal. El denso trae chunks "temáticamente cercanos" de otros artículos.

La pregunta no es cuál usar — es cómo COMBINARLOS. Y hay un problema técnico: BM25 da puntajes tipo 12.7 y el coseno da 0.83. Son escalas incomparables... ¿cómo se mezclan? (siguiente slide)

Pista: la respuesta es NO mezclar puntajes.

Reciprocal Rank Fusion: fusionar por PUESTOS, no por puntajes.

$$\text{RRF}(\text{doc}) = \sum 1 / (k + \text{puesto_del_doc_en_cada_lista}) \quad \text{con } k \approx 60$$

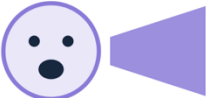
Cada buscador entrega su ranking. Un documento suma puntos según su PUESTO en cada lista (1º suma mucho, 10º casi nada). Se ordena por la suma y listo:

- **Inmune a las escalas:** los puestos son comparables aunque los puntajes no — 1º es 1º en cualquier escala. Ese era el problema de la slide anterior, resuelto.
- **Premia el consenso:** un chunk decente en AMBAS listas le gana a uno brillante en una sola — dos testigos independientes valen más que uno entusiasta.
- **Trivial de implementar:** son tres líneas de código (lo verán en el lab). Cero modelos nuevos, cero GPU extra.

Analogía: dos jurados con escalas distintas (uno califica 1-10, otro 1-100). No promedies notas — promedia PUESTOS.

RRF: le baja el volumen a los dos jurados

tres chunks candidatos, dos buscadores que hablan en escalas que no tienen nada que ver



¡12.7!

BM25
el que GRITA
habla en puntos: 12.7



...0.91...

DENSO
el que SUSURRA
habla en cosenos: 0.91

Cada jurado ordena los 3 chunks a su manera:

BM25 : A = 12.7 B = 4.2 C = —
DENSO: A = — B = 0.83 C = 0.91
(— = ni siquiera lo trajo en su top-k)

✗ Si PROMEDIAMOS puntajes...
los dos jurados hablan al tiempo — y uno grita

chunk	BM25 dice	DENSO dice	promedio
chunk A	12.7	—	6.35
chunk B	4.2	0.83	2.5
chunk C	—	0.91	0.45

Gana A — porque BM25 GRITÓ más.

El 0.91 del denso ni se oyó: en un promedio, 12.7 aplasta a 0.91 aunque los dos signifiquen “¡este es el mejor!” en su escala.

El error: sumar peras con kilómetros.
Un puntaje solo tiene sentido dentro de SU escala.

✓ Con RRF, solo cuentan los PUESTOS
a los dos se les baja el volumen: 1º es 1º, grite o susurre

chunk	puesto BM25	puesto DENSO	$\Sigma 1/(60+\text{puesto})$	RRF
chunk A	1º	—	$1/61 + 0$	0.016
chunk B	2º	2º	$1/62 + 1/62$	0.032
chunk C	—	1º	$0 + 1/61$	0.016

Gana B — el único que convenció a los DOS.

Mismo tamaño de letra para todos: un puesto no tiene unidades, así que puntos y cosenos por fin se pueden sumar.

Dos testigos tibios valen más que uno entusiasta.
RRF premia el consenso — y es inmune a las escalas.

Para recordar: **no promedies notas de jurados con escalas distintas — fusiona PUESTOS.**

Un segundo par de ojos.

Recuperar mucho con lo barato, reordenar poco con lo bueno: cross-encoders.

Bi-encoder vs. cross-encoder.

BI-ENCODER (EL DE SU BASE)

Vectoriza por separado

Pregunta y chunk se embeben CADA UNO por su lado; luego se compara con coseno.

- ✓ Los chunks se vectorizan UNA vez → buscar entre millones es barato.
- ✗ Nunca "lee" pregunta y chunk juntos: pierde matices.

CROSS-ENCODER (EL RERANKER)

Lee los dos JUNTOS

Recibe (pregunta, chunk) como UN par y produce un puntaje de relevancia leyéndolos con atención cruzada.

- ✓ Mucho más preciso: nota negaciones, condiciones, matices.
- ✗ Carísimo por par: imposible correrlo contra todo el corpus.

Conexión con S03: el cross-encoder es un modelo tipo BERT (encoder-only) que clasifica el PAR pregunta-chunk. El mismo transformer, en su tercer oficio del curso.

¿Barato-masivo o caro-preciso? Respuesta de ingeniería: LOS DOS, en cascada. (siguiente slide)

El embudo: recuperar mucho, reordenar poco.



La economía del embudo: lo barato (bi-encoder) filtra miles → decenas; lo caro (cross-encoder) solo lee esas decenas → entrega los mejores 5. Precisión de lujo a costo razonable. Es el mismo patrón de un proceso de selección: filtro de hojas de vida masivo, entrevista solo a los finalistas.

Arreglar la pregunta.

A veces el problema no es la búsqueda ni el ranking: es la consulta misma. Un LLM puede reescribirla.

La pregunta del usuario no es la mejor consulta.

- **Coloquial:** "¿me toca pagar si devuelvo eso?" — ni una palabra del chunk de "política de reembolsos".
- **Doble o compuesta:** "¿qué vacunas y qué papeles necesita mi gato para ir a Madrid?" — DOS preguntas; un solo embedding las promedia y no encuentra bien ninguna.
- **Demasiado específica:** "¿puedo fumigar un lote de 3 hectáreas en Urrao un martes?" — el corpus habla en general de fumigación; lo hiper-específico no matchea.

La idea de esta familia: usar un LLM ANTES de buscar, para transformar la consulta en una (o varias) que sí encuentren.
Costo: una llamada extra al LLM por consulta — latencia y plata. Por eso también se mide.



Cuatro técnicas: multi-query • HyDE • step-back • decomposition. Vamos una por una.

Multi-query: preguntar de varias formas.

El LLM genera 3-5 reformulaciones de la consulta; se busca con TODAS y se fusionan los resultados (RRF otra vez — ya la tienen).

Original: "¿me toca pagar si devuelvo eso?"

Reformulaciones del LLM: "¿la devolución de un producto tiene costo?" · "política de reembolsos y devoluciones" · "¿quién asume el envío de una devolución?"

- **Cuándo brilla:** consultas coloquiales o ambiguas — alguna reformulación cae cerca del vocabulario del corpus.
- **Su costo:** una llamada al LLM + k búsquedas. La más simple de la familia y la primera que deben probar.

HyDE: buscar con una respuesta imaginaria.

Hypothetical Document Embeddings: el LLM escribe una respuesta HIPOTÉTICA a la pregunta (sin corpus, puede tener errores — no importa) y se busca con el embedding de ESA respuesta, no con el de la pregunta.

¿Por qué funciona? Porque una pregunta y su respuesta VIVEN LEJOS en el espacio de embeddings ("¿me toca pagar...?" no se parece a un reglamento), pero una respuesta imaginaria y la respuesta real se PARECEN entre sí: misma forma, mismo vocabulario, mismo registro. Buscamos parecido-a-la-respuesta en vez de parecido-a-la-pregunta.

La paradoja pedagógica de HyDE: usamos una posible alucinación... para ENCONTRAR la verdad. El documento hipotético nunca llega al usuario — es solo el anzuelo de búsqueda; la respuesta final se genera con los chunks REALES recuperados.

- **Cuándo brilla:** preguntas cuyo estilo no se parece en nada al estilo del corpus (coloquial vs. técnico-legal).
- **Riesgo:** si la hipótesis se va por otro tema, la búsqueda se desvía con ella. Medir, como siempre.

Step-back y decomposition.

STEP-BACK

Dar un paso atrás

Transformar la pregunta hiper-específica en su versión GENERAL antes de buscar.

"¿puedo fumigar 3 ha en Urrao un martes?" → "¿condiciones y restricciones para fumigar cultivos?"

Recupera el marco general; el LLM luego aterriza al caso.

DECOMPOSITION

Partir en sub-preguntas

Una consulta compuesta se parte en preguntas atómicas; se busca CADA una y se juntan los contextos.

"¿vacunas y papeles para llevar mi gato a Madrid?" → (1) vacunas gato viaje UE · (2) documentos gato viaje UE

Cada sub-búsqueda matchea limpio.

Las cuatro técnicas comparten el patrón: gastar UNA llamada extra de LLM antes de buscar, para que la búsqueda (que es lo barato) encuentre mejor. Se combinan con hybrid y reranking sin conflicto.

Regla práctica de selección: coloquial → multi-query o HyDE • compuesta → decomposition • hiper-específica → step-back.

Resumen: qué arregla cada técnica y qué cuesta.

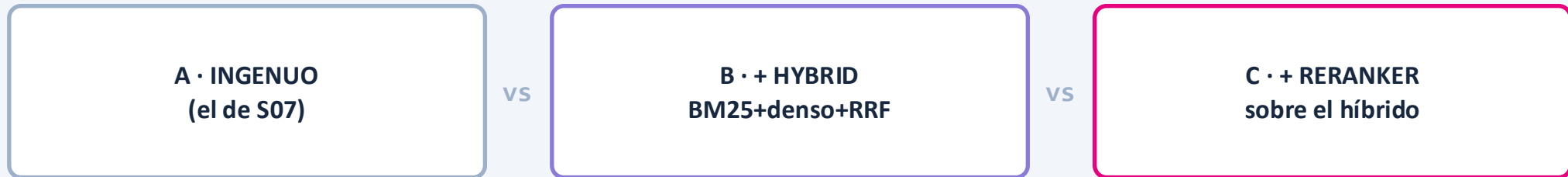
	Hybrid (BM25+denso+RRF)	términos exactos que el denso difumina	casi gratis (sin LLM extra)
	Reranking (cross-encoder)	ruido en el top-k	un modelo más; +latencia moderada
	Multi-query	consultas coloquiales/ambiguas	1 llamada LLM + k búsquedas
	HyDE	estilo de pregunta ≠ estilo del corpus	1 llamada LLM (genera hipótesis)
	Step-back / decomposition	preguntas hiper-específicas / compuestas	1 llamada LLM; contextos más largos

Para la entrega M3 (S10): su sistema debe integrar AL MENOS DOS técnicas avanzadas — elegidas por su diagnóstico, justificadas con su delta.

Cada técnica se gana su lugar.

El experimento: ingenuo vs. híbrido vs. híbrido+reranker — mismo eval set, mismo harness.

Un experimento controlado, tres sistemas.



```
harness(eval_set, A) vs harness(eval_set, B) vs harness(eval_set, C) → tabla de deltas
```

- Lo único que cambia entre A, B y C es el retrieval — mismo eval set, mismo juez, misma rúbrica: cualquier delta es atribuible a la técnica.
- Anoten también la latencia por consulta — una mejora de +0.05 que duplica el tiempo de respuesta puede no valer la pena para SU usuario.
- Y prueben sus consultas fallidas de S07 — el delta agregado importa, pero ver SU consulta rota por fin encontrando el chunk correcto es el momento de la clase.

Al notebook — Lab

Mejoren su retrieval y midan el delta

Sobre su corpus de S07: implementan BM25 y la fusión RRF (hybrid), agregan el cross-encoder reranker en cascada, y — opcional — multi-query. Corren el harness sobre los tres sistemas y llenan la tabla de deltas. Cierran con la prueba reina: sus consultas fallidas de la semana pasada contra el sistema C.

```
bm25 + rrf → reranker → harness × 3 → tabla de deltas → S08_Lab_RAG_avanzado.ipynb
```

Clase grabada: pausen el video aquí, trabajen el lab con su equipo (~35 min) y vuelvan para el cierre.

Cómo leer su tabla de deltas.

- **Si $B > A$:** su corpus tiene vocabulario exacto que el denso difuminaba — el híbrido paga. Es lo típico en legal, e-commerce (SKUs) y normativa.
- **Si $C > B$:** su problema era ruido en el top-k — el reranker limpia. Suele ser la mejora más consistente de las dos.
- **Si algo NO mejoró (o empeoró):** no es fracaso — es diagnóstico: esa técnica no ataca SU fallo. Repórtenlo con su latencia: "no valió su costo" es una conclusión de ingeniería perfectamente publicable.
- **Con 10-12 ejemplos, deltas pequeños son ruido:** un salto de 6/12 a 7/12 puede ser azar. Miren caso por caso QUÉ cambió — el detalle vale más que el promedio.

Toda técnica cobra: calidad vs. latencia vs. costo.

	Ingenuo	1 embedding + 1 búsqueda	~rápido
	+ Hybrid	+ BM25 + fusión	+ milisegundos
	+ Reranker	+ cross-encoder sobre 30 pares	+ décimas de segundo
	+ Multi-query / HyDE	+ 1 llamada LLM antes de buscar	+ segundos

La pregunta de diseño no es "¿cuánto sube el harness?" sino "¿cuánto sube POR segundo de latencia y peso de infraestructura?". Su usuario de salud pública esperando una respuesta urgente no piensa igual que su usuario de e-commerce curioseando. El scorecard de M3 pide reportar ambos.

S09: parcial + checkpoint público.

1

PARCIAL (15%, individual, 45 min, sin computador): cubre M1, M2 y RAG inicial. OJO: hagan el taller de nivelación A CONCIENCIA (tres sesiones, escribiendo antes de mirar soluciones) y el parcial se sentirá como una ronda más.

2

CHECKPOINT PÚBLICO (5 min por equipo): estado del proyecto — su scorecard del baseline, el delta del RAG con las técnicas de hoy, y el problema más duro que tienen abierto.

3

Suban a sus repo la tabla de deltas del lab de hoy — es insumo directo de la entrega M3 (S10) y del checkpoint.

El ingenuo era el piso. Hoy midieron cuánto subieron.

Hybrid para lo exacto, reranking para el ruido, query transformation para las preguntas difíciles — y cada una justificada con un delta, no con una moda. La próxima semana: parcial y checkpoint. En S10: agentic, RAGAS y la entrega M3.